

Lecture: 0111

Register Files & State Machines

The Storage at the Heart of a CPU

How a CPU reads two operands and writes one result every cycle — and how FSMs control it all.

CSA222: Modern Computer Architecture

1. Finite State Machines

The universal model of sequential logic: a set of states, rules to move between them, and outputs. This is how a CPU 'thinks'.

What Is a Finite State Machine?

A circuit that is always in exactly ONE of a finite set of states, and moves between them on rules.

THREE INGREDIENTS

1

States

A finite list of situations the machine can be in (e.g. IDLE, RUNNING, DONE).

2

Transitions

Rules for moving between states, based on current state + inputs.

3

Outputs

What the machine produces in each state (or during each transition).

EVERYDAY EXAMPLE

A traffic light is an FSM.

States: GREEN, YELLOW, RED.

Transitions: timed (green→yellow→red→green).

Outputs: which lamp is lit.

A vending machine, an elevator, a game character's AI, a network protocol — all FSMs. This model is everywhere.

Two Flavours: Moore vs Mealy

The difference is simply WHERE the output comes from.

MOORE MACHINE

Output depends on the STATE only

The output is written inside each state circle

It changes only when the state changes

Simpler to reason about, and glitch-free

May need more states to do the same job

Output appears one cycle after the input

MEALY MACHINE

Output depends on STATE + INPUT

The output is written on each transition arrow

It can react to an input within the same cycle

Often needs fewer states

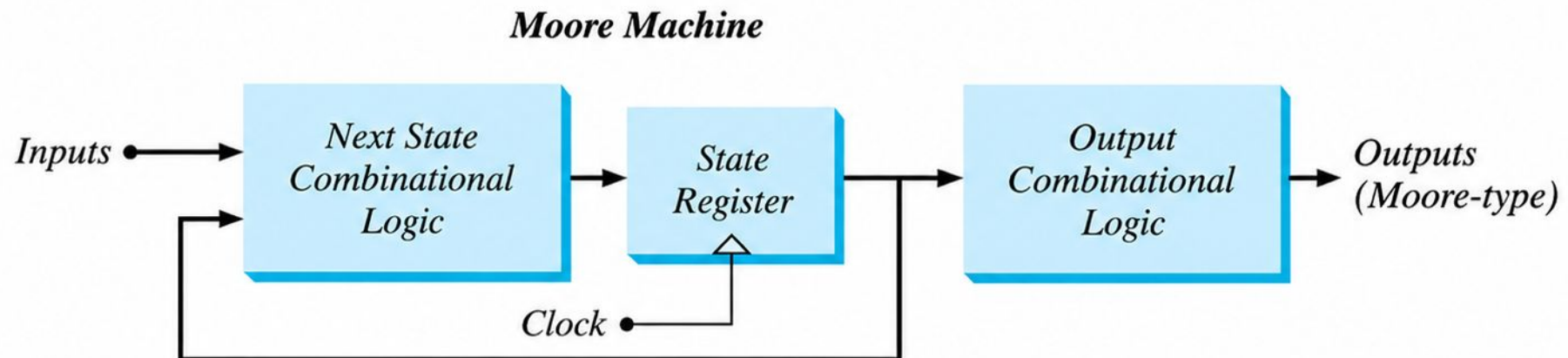
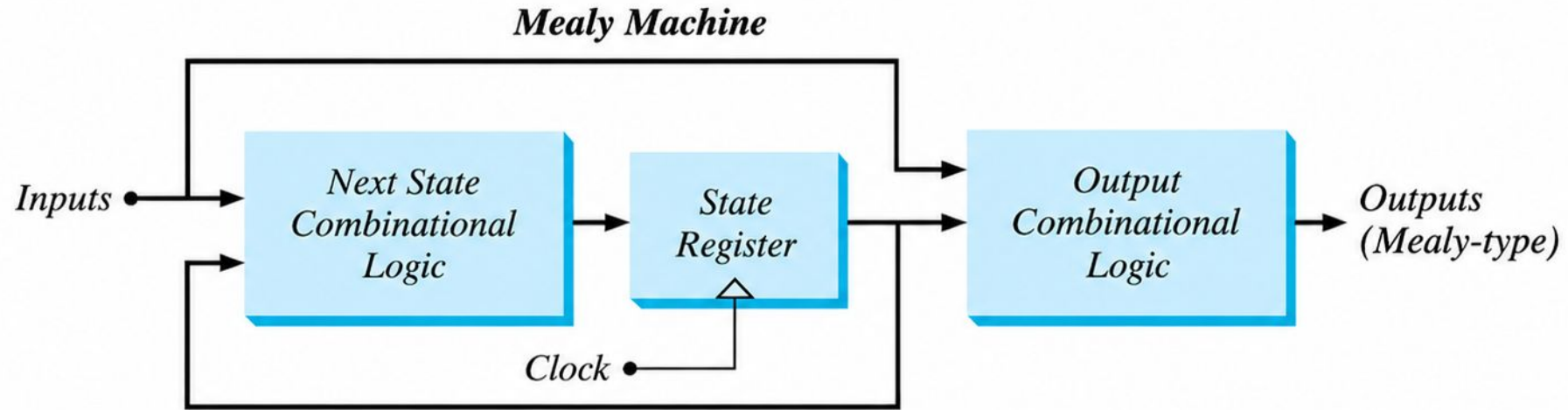
Output can glitch when the input glitches

Output appears in the same cycle as the input

Rule of thumb: Moore = 'output in the circle', Mealy = 'output on the arrow'. Both compute the same things.

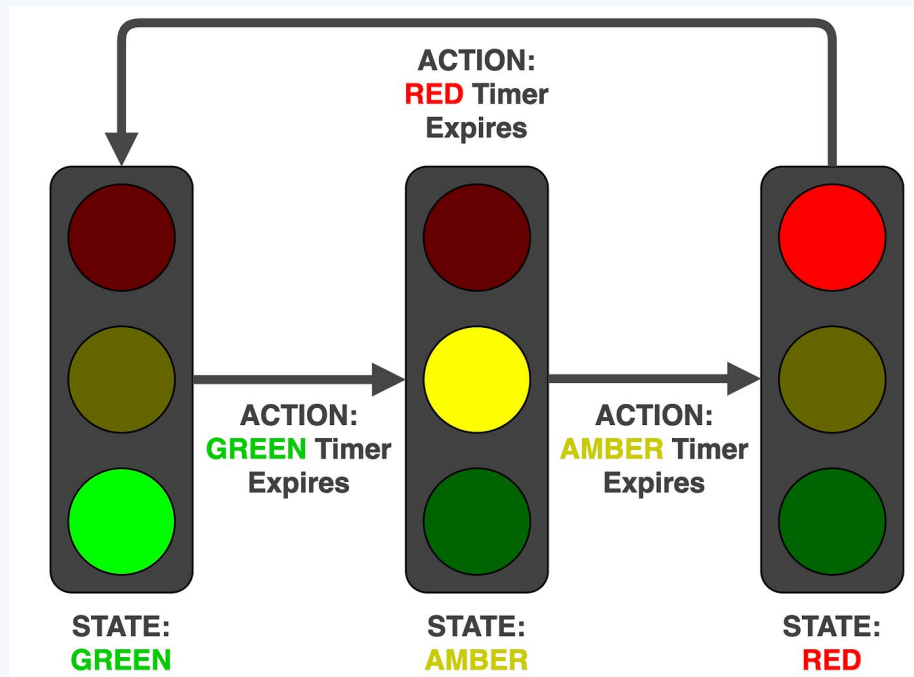
The Anatomy of Any FSM

Every sequential circuit is this same three-part structure.



The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.



HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

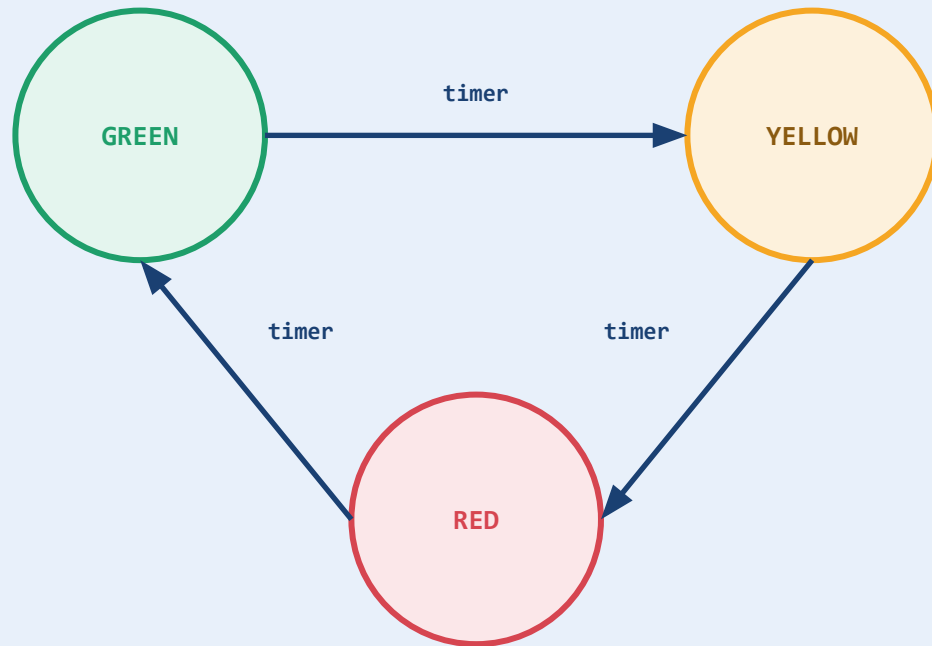
Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.

TRAFFIC LIGHT — A THREE-STATE MOORE MACHINE



HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

The State Table

The same FSM, written as a table — the bridge from diagram to hardware.

Current State	Input	Next State	Output
GREEN	expired	YELLOW	Go
YELLOW	expired	RED	Slow
RED	expired	GREEN	Stop
any	not expired	(same)	(hold)

Why a table? It lists every (state, input) → (next state, output). Encode the states as bits, and each column becomes a truth table you minimise with K-maps (L2) into gates.

FSM Design Flow

Six steps from a word problem to a working circuit — always the same recipe.

1**Understand**

List the states and what triggers each transition.

2**State diagram**

Draw circles and arrows — the visual spec.

3**State table**

Tabulate every state/input → next-state/output.

4**Encode states**

Assign a bit pattern to each state (00, 01, 10...).

5**Derive logic**

K-map each next-state & output bit into gates.

6**Build**

Flip-flops for the state register + that logic.

FSMs Run the World

Once you know the pattern, you'll see state machines in everything you build.



The CPU control unit

Fetch → Decode → Execute → Writeback is literally an FSM. It sequences every instruction (Module 4).



Communication protocols

TCP connection states, USB handshakes, traffic-light controllers — all specified as FSMs.



Software & games

UI flows, game AI ('patrol → chase → attack'), regex engines, parsers — FSMs in code, not gates.

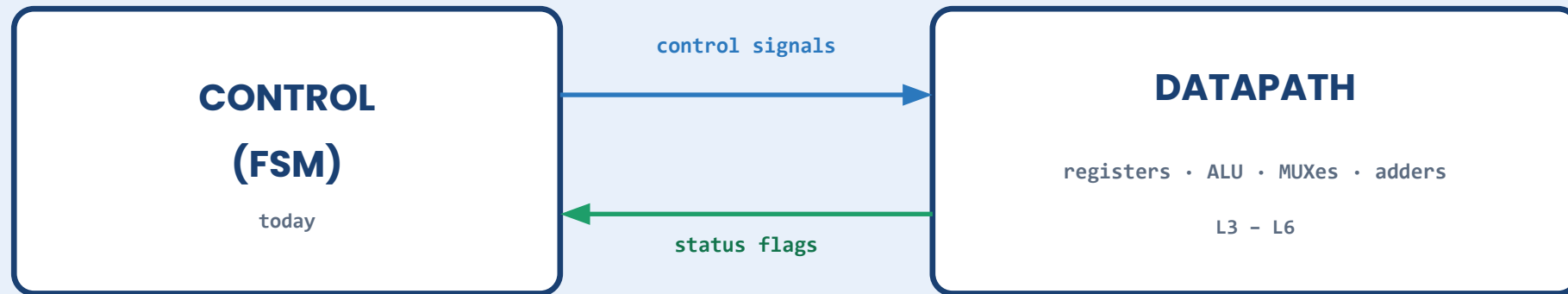


Embedded control

Washing machines, elevators, vending machines, engine controllers — physical FSMs everywhere.

The Big Picture: Datapath + Control

Everything in this course so far now snaps together into a computer.



The control unit is an FSM (today). The datapath is registers + ALU + MUXes (L3-L6). Together they're a CPU.

This is exactly where Module 3 & 4 pick up — you now have every building block a processor needs.

Where We Left Off

L5–L6 gave us flip-flops, registers, and counters. Now we organise them.



Flip-flops

Edge-triggered 1-bit memory, safe under a clock (L5).



Registers

n flip-flops sharing a clock = an n -bit word store (L6).



Counters & FSMs

Structures that count and follow state-based rules (L6).



Today: organise them

Pack 32 registers into a file; formalise FSM control.

A CPU has dozens of registers accessed constantly. How do you read two and write one, all in one cycle?

Today's Roadmap

The CPU's fastest storage, and the machines that control it.

1

Register Files

32 registers as one block: 1 write port, 2 read ports — the RISC standard.



2

Finite State Machines

The formal model of controlled sequential behaviour.



3

Moore vs Mealy

Two ways to build an FSM — and a '1011' detector in both.

Why it matters: the register file is the structure a CPU touches on nearly every instruction — and an FSM is what tells it when.

From Registers to a Register File

A CPU needs many registers, accessed together — so we pack them into one addressable block.

THE NEED

A single instruction like `add R1, R2, R3` must:

- READ two source registers (R2, R3)
- WRITE one destination register (R1)
- ...all in ONE clock cycle.

A register file is a bank of registers with address-based access — like a tiny, ultra-fast memory that lives inside the CPU.

THE STANDARD

MIPS & RISC-V: 32 registers, each 32 bits wide.

Ports: 2 read + 1 write, every cycle. That's exactly enough for a two-operand, one-result instruction.

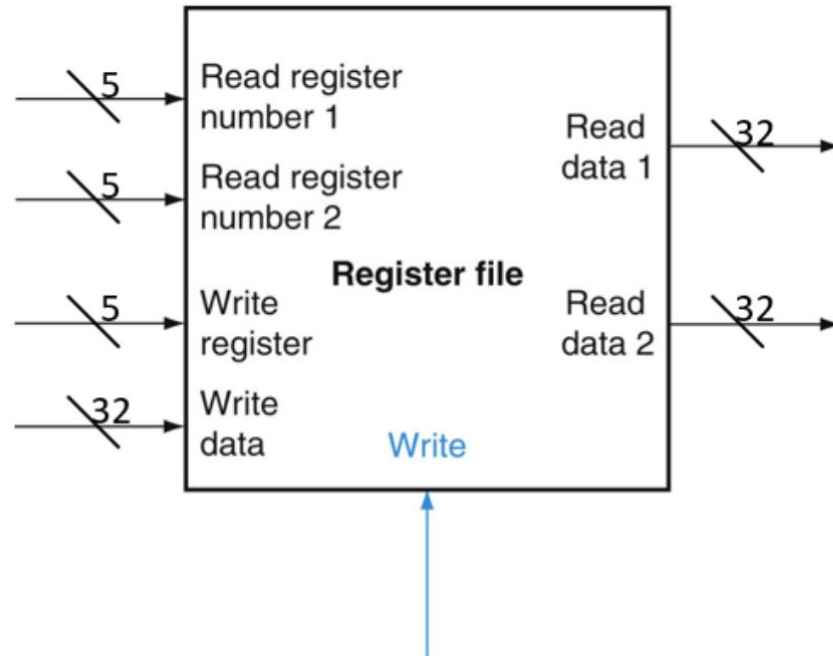
This exact structure feeds the ALU you'll wire up in Module 4 (L13–L14).

2. Register Files

A bank of registers with addressed access: a decoder picks where to write, MUXes pick what to read.

Anatomy of a Register File

Three ports in, addresses select which registers each port touches.



THREE PORTS

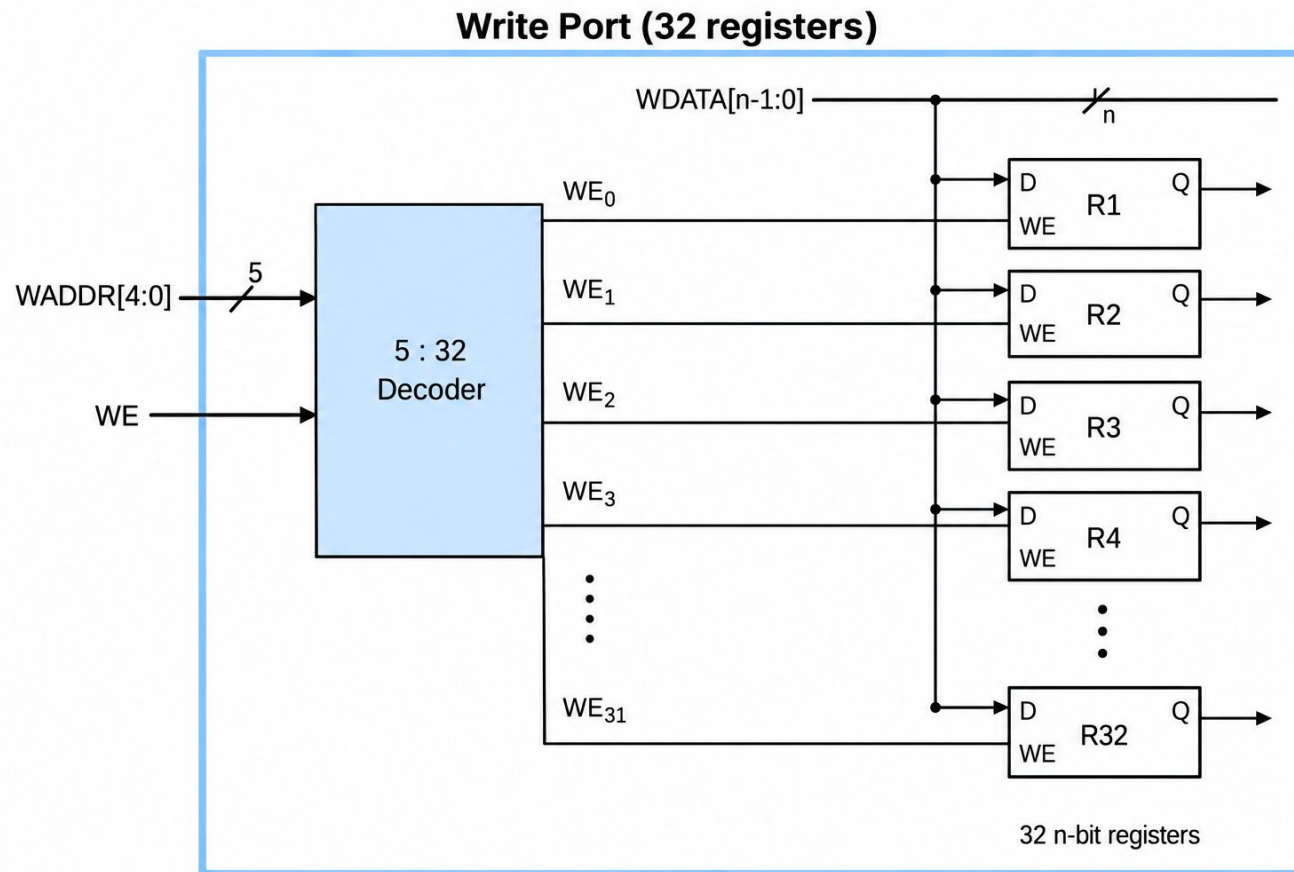
2 READ ports (blue):
each takes an address, outputs that register's value.

1 WRITE port (amber):
address + data + write-enable, updates one register on the clock edge.

Reads are instant (combinational); writes wait for the clock. That asymmetry is key — next slides.

The Write Port: Decoder + Enable

An address decoder activates exactly one register's clock-enable.



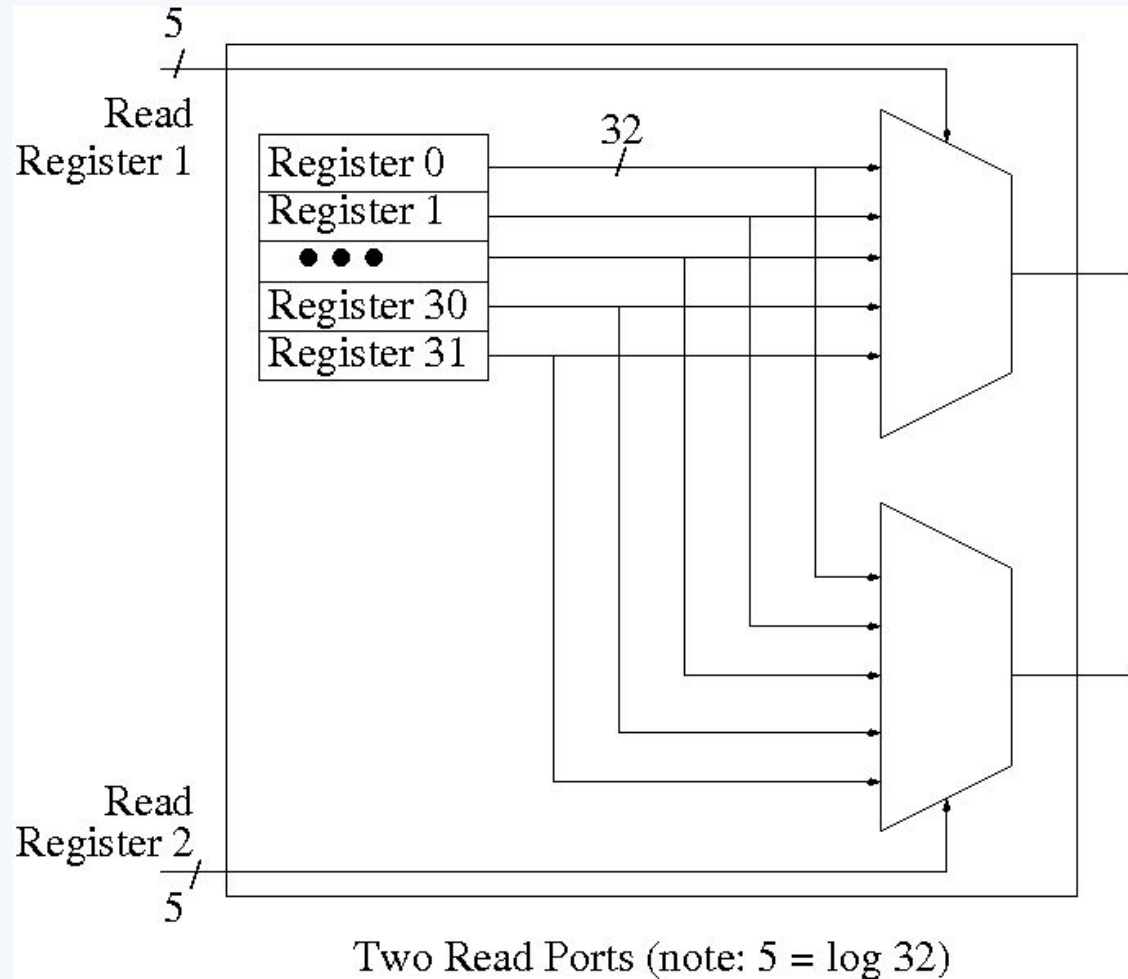
HOW A WRITE WORKS

1. The write address goes into a 5-to-32 decoder.
2. The decoder raises exactly ONE register's enable line (one-hot).
3. If $WE = 1$, that one register captures $wdata$ on the clock edge.

All other registers ignore the bus — their enables are 0.

The Read Ports: One Big MUX Each

Each read address drives a 32:1 MUX that selects one register onto the output.



WHY A MUX

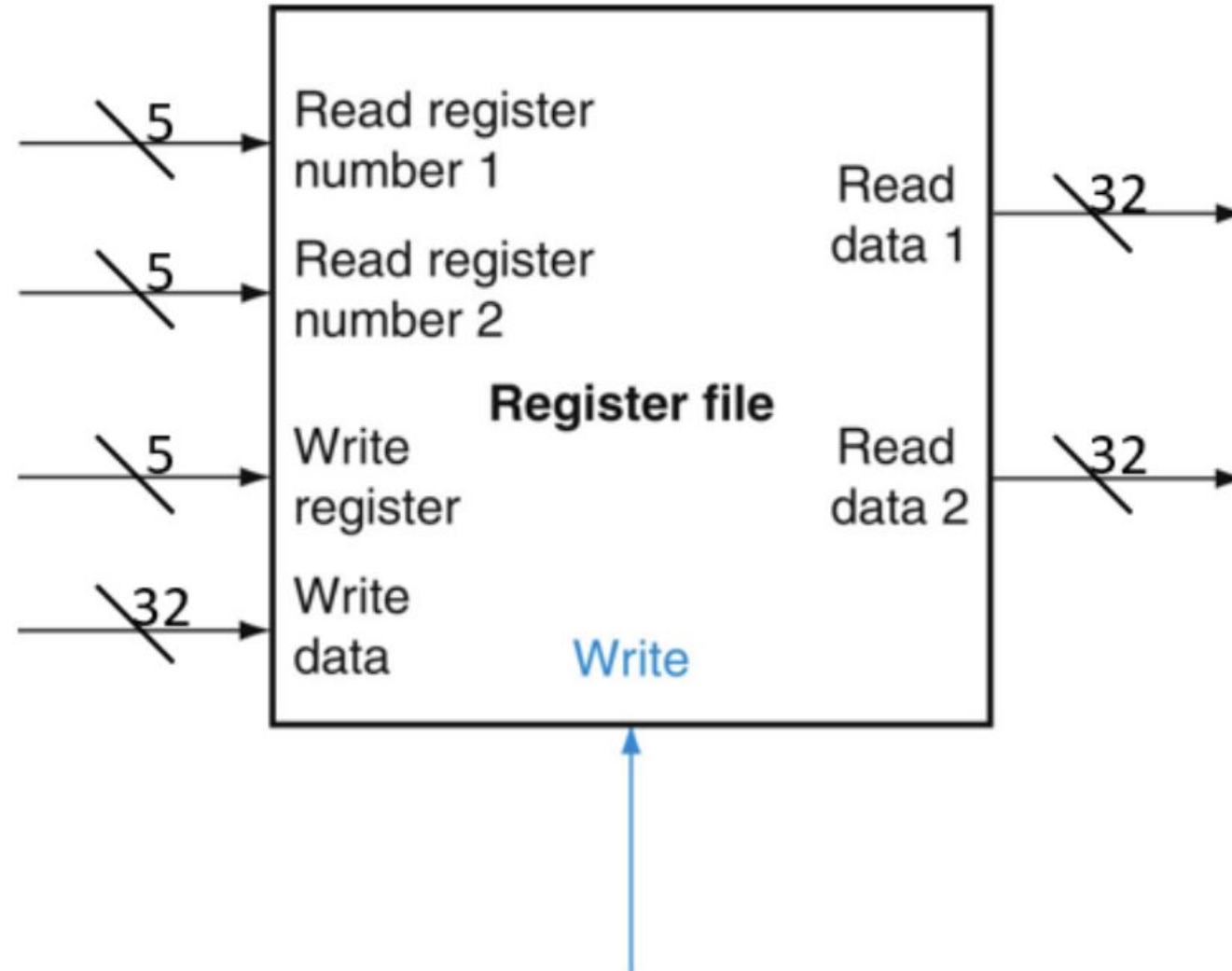
Every register's output is wired to **BOTH** read MUXes at once.

The read address is the select code — it picks which of the 32 register outputs flows to that port.

Two independent MUXes = two independent reads, simultaneously. Recall the MUX from L3.

Two Reads + One Write, Every Cycle

The full picture: one instruction's worth of register access in a single diagram.



Reads Are Instant, Writes Wait for the Clock

This asymmetry is the single most important fact about a register file.

READ = COMBINATIONAL

No clock needed

- Put an address on a read port...
- ...the data appears after just gate delay.
- Both read ports work this way, at once.
- So the ALU gets its operands immediately.

WRITE = SYNCHRONOUS

Happens on the clock edge

- Address + data + WE are set up early...
- ...the register updates only at the edge.
- This prevents races and half-written values.
- The new value is readable next cycle.

Why it works: read the old values combinatorially at the start of a cycle, compute, then write the result at the edge — read-before-write, safely, in one cycle.

The Register File in a Real CPU

This isn't theory — it's the exact structure inside RISC-V and MIPS chips.



RISC-V: 32 registers

x0-x31, each 32 or 64 bits. x0 is hardwired to zero — reads always give 0, writes are ignored.



Always 2 read + 1 write

Matches R-type instructions (add rd, rs1, rs2): read two sources, write one destination per cycle.



Fastest storage tier

Register file → L1 cache → L2 → RAM. Registers are a fraction of a nanosecond away from the ALU.



Feeds the datapath

In Module 4, the read ports drive the ALU and the write port catches its result — the core loop.

Common Mistakes to Avoid

The slips that cost marks — and cause real bugs.



Clocking the read port

Register-file reads are combinational. Clocking them adds a needless cycle of latency.



Mixing Moore & Mealy notation

Moore: output in the circle (S1/1). Mealy: output on the arrow (in/out). Don't blend them.



Forgetting write-enable

Without WE, a register is overwritten every cycle. The decoder picks WHERE; WE picks WHETHER.



Miscounting detector states

Moore needs a state per output value → often one more than the Mealy version. Count carefully.



Resetting a detector on match

After '1011', the trailing '1' may start a new match. Go to the right prefix state, not S0.



Tri-state with two drivers

On a tri-state read bus, enabling two buffers at once causes bus contention — a hardware fault.

Key Takeaways

Five sentences — the CPU's storage and its controller.

- 01** A register file packs many registers (RISC: 32×32 -bit) into one block with 2 read ports and 1 write port.
- 02** The write port uses a decoder + write-enable to update exactly one register on the clock edge (synchronous).
- 03** The read ports are MUX-based and combinational — two operands appear instantly, no clock needed.
- 04** An FSM = states + inputs + transitions + outputs, built as next-state logic → state register → output logic.
- 05** Moore output = $f(\text{state})$ (in the circle); Mealy output = $f(\text{state}, \text{input})$ (on the arrow) — Mealy often uses fewer states.

Practice & Homework

Design register files and state machines.

01

8×16 Register File

Design a register file with 8 registers of 16 bits. Show the write-port decoder logic (3→8).

02

Mealy '10' Detector

Draw the state diagram for a Mealy FSM that outputs 1 whenever it sees the pattern '10'.

03

Mealy → Moore

Convert a given 3-state Mealy FSM into an equivalent Moore FSM. How many states now?

EXIT TICKET

1. In a register file, why is the read combinational but the write synchronous?
2. How many states does a Moore '1011' detector need, and why one more than Mealy?

3. Trace the Mealy detector on input '101011' — when does the output pulse?

END OF L07

Storage, and the machine to run it.

NEXT — L08

Memory & the Memory Hierarchy — SRAM vs DRAM, how RAM is organised, and why your CPU waits on memory more than anything else.